TECHNICAL FIELD REFERENCE & ARCHITECTURE GUIDE

Comprehensive Django & Full-Stack Guide

From Virtual Environments and Database Engineering to Tailwind Pipelines and Render Deployment

1. Environment Setup & Core Architecture

1. Create virtual environment { python -m venv task_env }

A virtual environment encapsulates all Python binaries and third-party packages in an isolated local directory, preventing cross-project library conflicts.

```
python -m venv task_env
```

2. Activate Environment { task_env/scripts/activate / (task_env\Scripts\activate)task_env/scripts/activate.bat }

Run the respective script for your operating system and shell:

```
# Windows Command Prompt
task_env\Scripts\activate.bat

# Windows PowerShell
task_env\Scripts\Activate.ps1

# Git Bash / macOS / Linux
source task_env/Scripts/activate
```

PowerShell Script Execution Policy Error:

```
PS F:\Ridan All\Ridan\Phitron\Development-With-Python\Django\task-management>
task_env/scripts/activate
task_env/scripts/activate : File F:\Ridan All\Ridan\Phitron\Development-With-
Python\Django\task-management\task_env\scripts\Activate.ps1 cannot be loaded because
running scripts is disabled on this system. For more information, see
about_Execution_Policies at https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ task_env/scripts/activate
+ ~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
```

Resolution: If this error occurs, run this command in PowerShell:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

3. Install Django { pip install Django }

Install the Django web framework inside the active environment:

```
pip install Django
```

To check Django installed { pip list }:

```
pip list
```

2. Django Project & App Construction

4. Create a Django project { django-admin startproject task_management . }

```
django-admin startproject task_management .
```

Note: The trailing period tells Django to initialize the project files in the current folder without creating an extra nested folder level.

5. Create application { django-admin startapp tasks [appName] }

```
django-admin startapp tasks
```

Register the newly generated app within `task_management/settings.py` under `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    # Built-in apps...
    'tasks',
]
```

6. Work with view on tasks

Define request-handling functions in `tasks/views.py`:

```
from django.shortcuts import render
from django.http import HttpResponse

def home(request):
    return HttpResponse("Task Manager Active")
```

7. Then url in task_management

Open `task_management/urls.py` and bind the application routes:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('tasks.urls')),
]
```

8. Run server { python manage.py runserver }

```
python manage.py runserver
```

9. Create an urls.py in each app then define url here for this app then include it in main url in main task_management project

Create `tasks/urls.py`:

```
from django.urls import path
from .views import home

urlpatterns = [
    path('', home, name='home'),
]
```

3. Dependency Locking & IDE Configuration

10. Create a requirement.txt for making this virtual environment portable { pip freeze > requirements.txt }

```
pip freeze > requirements.txt
```

After that, whenever we move this project to another drive or computer, we can delete the environment directory, recreate a clean one using `python -m venv task_env`, activate it using `task_env/scripts/activate`, and then install requirements:

```
pip install -r requirements.txt
```

11. Ctrl + Shift + P (virtual env activation)

In VS Code, press `Ctrl + Shift + P`, search for **Python: Select Interpreter**, and select the interpreter located inside `.\task_env\Scripts\python.exe`.

4. Static Assets & Tailwind CSS Build Chain

12. Create template folder in tasks app

Create the folder `tasks/templates/tasks/` for HTML files.

13. Create static folder in root for image, css, js

Add this code in `settings.py` of root:

```
STATICFILES_DIRS = [
    BASE_DIR / 'static',
]
```

14. Create package.json file in root and paste

```
{
  "devDependencies": {
    "autoprefixer": "^10.4.20",
    "postcss": "^8.4.49",
    "tailwindcss": "^3.4.16"
  },
  "scripts": {
    "build:tailwind": "npx tailwindcss -i ./static/css/tailwind.css -o
./static/css/output.css --minify",
    "watch:tailwind": "npx tailwindcss -i ./static/css/tailwind.css -o
./static/css/output.css --watch"
  }
}
```

This code installs Tailwind CSS in our project. Then open the terminal in VS Code and run:

```
npm install
npx tailwindcss init
```

15. Modify tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: [
    "./templates/**/*.html",
    "./**/templates/**/*.html",
    "./static/**/*.js",
    "./**/*.py",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

16. Create a file named: postcss.config.js and add some code here

```
module.exports = {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
};
```

17. Add input.css in static folder css folder

Create `static/css/tailwind.css` (or `input.css`):

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

18. Then run: npm run build:tailwind / npm run watch:tailwind

```
npm run watch:tailwind
# or for production:
npm run build:tailwind
```

5. Front End & Template Rendering

19. Explore Django template tags

Common tags for dynamic presentation:

```
{% extends 'base.html' %}
{% load static %}
<link rel="stylesheet" href="{% static 'css/output.css' %}">

{% for task in tasks %}
    <h3>{{ task.title }}</h3>
    {% if task.is_completed %}
        <span>Completed</span>
    {% else %}
        <span>Pending</span>
    {% endif %}
{% empty %}
    <p>No tasks available.</p>
{% endfor %}
```

6. PostgreSQL Connection & Models

20. Postgres connection in Django

Create a database then add database configuration in `settings.py` in `task_management`:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'task_management',
        'USER': 'postgres',
        'PASSWORD': '12345',
        'HOST': 'localhost',
        'PORT': '5432'
    }
}
```

Install package for using postgres { psycopg-binary }: `pip install psycopg-binary`. If you get an error after migration, use: `pip install psycopg2-binary`.

21. Add Task class in task app's model.py

```
from django.db import models

class Task(models.Model):
    title = models.CharField(max_length=100)
    description = models.TextField()
    due_date = models.DateField()
    is_completed = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

Run commands in terminal:

```
python manage.py makemigrations
python manage.py migrate
```

For fake data:

```
python populate_db.py
```

22. Then create Task details class in same file

```
class TaskDetail(models.Model):
    task = models.OneToOneField(Task, on_delete=models.CASCADE, related_name='details')
    priority = models.CharField(max_length=50, default='Medium')
    notes = models.TextField(blank=True, null=True)
```

Then run commands:

```
python manage.py makemigrations
python manage.py migrate
```

23. Python manage.py shell (from test database tables values)

```
python manage.py shell

from tasks.models import Task
t = Task(title="Low Perioty task", description="This is first task", due_date="2026-12-
25")
Project.objects.create(name="Demo Project", start_date="2026-12-15")
project=Project.objects.all()
project.first().id
t.title
t.save()
```

7. ORM Profiling, Migrations Reset & Seeding

24. Django ORM : django debug toolbar

Install and configure Django Debug Toolbar for query performance tracking:

```
pip install django-debug-toolbar
```

In `settings.py`, add `'debug_toolbar'` to `INSTALLED_APPS`, `'debug_toolbar.middleware.DebugToolbarMiddleware'` at the top of `MIDDLEWARE`, and set `INTERNAL_IPS = ['127.0.0.1']`.

25. Modify models.py and delete database from pgAdmin and delete all file from migrations instead __init__.py then create database with same database name and then makemigration and migrate

When resetting development schemas:

- Drop and recreate the database in pgAdmin.
- Remove all migration files inside each app's `migrations/` directory *except* `__init__.py`.
- Execute `python manage.py makemigrations` followed by `python manage.py migrate`.

26. Insert some data in database using populate_db.py file

For this we install the Faker package { faker python }: `pip install Faker`.

```
python populate_db.py

# or via shell:
1. python manage.py shell
2. from populate_db import populate_db
3. populate_db()
```

27. Queryset api or lookupapi for get, filter, and many more functions

Operation	Django ORM Syntax	Description
Fetch All	`Task.objects.all()`	Retrieves all table records as a QuerySet.
Filter Exact	`Task.objects.filter(is_completed=False)`	Returns matching subset.
Case-Insensitive Match	`Task.objects.filter(title__icontains='urgent')`	Matches substring in text field.
Comparison Lookup	`Task.objects.filter(due_date__gte='2026-12-01')`	Greater than or equal to specified date.
Fetch Single Object	`Task.objects.get(id=1)`	Returns a single object; raises exception if missing.

8. Version Control & Ignore Configuration

Create `.gitignore` in PowerShell:

```
New-Item .gitignore
```

```
task_env/
venv/
env/
__pycache__/
*.pyc
*.sqlite3
db.sqlite3
.env
staticfiles/
media/
```

9. Production Deployment on Render

Deploy to Render (https://render.com/), provision PostgreSQL database, and copy the External Database URL.

Update `settings.py`:

```
ALLOWED_HOSTS = ['*']

CSRF_TRUSTED_ORIGINS = [
    'https://*.onrender.com',
    'http://127.0.0.1:8000',
    'http://localhost:8000',
]

import dj_database_url

DATABASES = {
    'default': dj_database_url.config(
        default='postgresql://event_db_k37v_user:ofP1KvFjw25Z0vl68Wqk1hXYokFT6TCy@dpg-
dbo0ctijnfac73diuieg-a.oregon-postgres.render.com/event_db_k37v',
        conn_max_age=600
    )
}
```

Install deployment packages and lock requirements:

```
pip install dj-database-url psycopg2-binary
pip freeze > requirements.txt
```